

Android 第八章第四节教学讲义（Fragment 实战与记账本案例）

各位同学，今天咱们重点学习**Fragment（碎片）**这个核心组件，包括它的静态注册、动态注册和生命周期，还会用 Fragment 改进启动引导页，最后通过一个"记账本"案例，把 Fragment 和之前学的列表控件结合起来，实现添加账单、展示账单等功能。内容实用，代码为主，保证小白也能跟着做！

一、Fragment 基础：什么是 Fragment？

Fragment 可以理解为"迷你 Activity"，它有自己的布局和生命周期，能嵌入到 Activity 中使用。优点是**灵活复用**（比如在手机上单页，在平板上多页），让界面设计更灵活。

二、Fragment 静态注册：在布局中直接声明

静态注册是在 Activity 的布局文件中直接写 Fragment 标签，简单但不够灵活（不能动态替换）。

步骤 1：创建 Fragment 类

```
public class StaticFragment extends Fragment {  
    // 加载Fragment布局  
    @Nullable  
    @Override  
    public View onCreateView(@NonNull LayoutInflater inflater,  
                             @Nullable ViewGroup container,  
                             @Nullable Bundle savedInstanceState) {  
        // 加载布局文件（res/layout/fragment_static.xml）  
        return inflater.inflate(R.layout.fragment_static, container, false);  
    }  
}
```

步骤 2：创建 Fragment 布局（fragment_static.xml）

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:background="#E0F7FA">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="这是静态注册的Fragment"
        android:textSize="18sp" />
</LinearLayout>
```

步骤 3：在 Activity 布局中静态注册

在 Activity 的布局文件（如 activity_static_fragment.xml）中添加：

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <!-- 静态注册Fragment：通过name指定Fragment类 -->
    <fragment
        android:id="@+id/fragment_static"
        android:name="com.example.myapplication.StaticFragment" <!-- 完整类名 -->
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
</LinearLayout>
```

步骤 4：在 Activity 中使用

```
public class StaticFragmentActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_static_fragment);
        // 无需额外代码，布局中已声明，运行即可看到Fragment
    }
}
```

```
}  
}
```

三、Fragment 生命周期：碎片的 "一生"

Fragment 的生命周期和 Activity 类似，但多了一些与 Activity 交互的方法，核心方法如下：

方法名	作用
onAttach()	与 Activity 关联时调用
onCreate()	Fragment 创建时调用
onCreateView()	加载布局，返回 Fragment 的视图
onActivityCreated()	宿主 Activity 创建完成后调用
onStart()	可见但不可交互
onResume()	可见且可交互
onPause()	失去焦点但可见
onStop()	不可见
onDestroyView()	视图被销毁（布局被移除）
onDestroy()	Fragment 销毁
onDetach()	与 Activity 解除关联时调用

实战：打印生命周期日志

在 Fragment 中重写这些方法，添加日志：

```
public class LifecycleFragment extends Fragment {  
    private static final String TAG = "FragmentLifecycle";  
    @Override  
    public void onAttach(@NonNull Context context) {  
        super.onAttach(context);  
        Log.d(TAG, "onAttach: 与Activity关联");  
    }  
}
```

```

}
@Override
public void onCreate(@Nullable Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    Log.d(TAG, "onCreate: Fragment创建");
}
@Nullable
@Override
public View onCreateView(...) {
    Log.d(TAG, "onCreateView: 加载布局");
    return inflater.inflate(R.layout.fragment_lifecycle, container, false);
}
@Override
public void onActivityCreated(...) {
    super.onActivityCreated(savedInstanceState);
    Log.d(TAG, "onActivityCreated: Activity创建完成");
}
// 其他生命周期方法类似，添加Log即可
}

```

运行后切换页面或旋转屏幕，观察 Logcat 中的日志，就能直观看到生命周期的变化。

四、Fragment 动态注册：代码中灵活控制

动态注册是在代码中通过 `FragmentManager` 添加或替换 `Fragment`，更灵活（比如点击按钮切换页面）。

步骤 1：创建两个 `Fragment`（用于切换）

```

// 第一个Fragment
public class FragmentA extends Fragment {
    @Nullable
    @Override
    public View onCreateView(...) {
        return inflater.inflate(R.layout.fragment_a, container, false);
    }
}
// 第二个Fragment

```

```
public class FragmentB extends Fragment {
    @Nullable
    @Override
    public View onCreateView(...) {
        return inflater.inflate(R.layout.fragment_b, container, false);
    }
}
```

步骤 2：Activity 布局（含切换按钮和 Fragment 容器）

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <Button
        android:id="@+id/btn_switch"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="切换Fragment" />
    <!-- Fragment容器：用FrameLayout -->
    <FrameLayout
        android:id="@+id/container"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
</LinearLayout>
```

步骤 3：在 Activity 中动态管理 Fragment

```
public class DynamicFragmentActivity extends AppCompatActivity {
    private boolean isShowingA = true; // 标记当前显示的Fragment
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_dynamic_fragment);
        // 初始显示FragmentA
        getSupportFragmentManager().beginTransaction()
            .add(R.id.container, new FragmentA()) // 添加到容器
    }
}
```

```

        .commit();
// 点击按钮切换Fragment
findViewById(R.id.btn_switch).setOnClickListener(v -> {
    // 开启事务
    FragmentTransaction transaction = getSupportFragmentManager().beginTransaction();
    if (isShowingA) {
        // 替换为FragmentB
        transaction.replace(R.id.container, new FragmentB());
    } else {
        // 替换为FragmentA
        transaction.replace(R.id.container, new FragmentA());
    }
    isShowingA = !isShowingA;
    transaction.commit(); // 提交事务
});
}
}

```

核心 API: `FragmentManager.beginTransaction()` 开启事务，`add()/replace()/remove()` 操作 `Fragment`，`commit()` 提交事务。

五、Fragment 改进启动引导页

之前用 `ViewPager` 实现引导页，现在用 `Fragment+ViewPager` 实现，更灵活（每个引导页逻辑独立）。

步骤 1：创建 3 个引导页 `Fragment`

```

public class GuideFragment extends Fragment {
    private int mResId; // 图片资源ID
    // 传入图片资源的方法
    public static GuideFragment newInstance(int resId) {
        GuideFragment fragment = new GuideFragment();
        Bundle args = new Bundle();
        args.putInt("resId", resId);
        fragment.setArguments(args);
        return fragment;
    }
}

```

```

@Override
public void onCreate(@Nullable Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    if (getArguments() != null) {
        mResId = getArguments().getInt("resId");
    }
}
@Nullable
@Override
public View onCreateView(...) {
    View view = inflater.inflate(R.layout.fragment_guide, container, false);
    ImageView iv = view.findViewById(R.id.iv_guide);
    iv.setImageResource(mResId); // 设置图片
    return view;
}
}

```

步骤 2：最后一页引导页（带 "开始" 按钮）

```

public class LastGuideFragment extends Fragment {
    @Nullable
    @Override
    public View onCreateView(...) {
        View view = inflater.inflate(R.layout.fragment_last_guide, container, false);
        // 点击按钮跳转到首页
        view.findViewById(R.id.btn_start).setOnClickListener(v -> {
            startActivity(new Intent(getContext(), MainActivity.class));
            getActivity().finish(); // 关闭引导页
        });
        return view;
    }
}

```

步骤 3：ViewPager 适配器（管理 Fragment）

```

public class GuidePagerAdapter extends FragmentPagerAdapter {
    private List<Fragment> mFragments;
    public GuidePagerAdapter(FragmentManager fm, List<Fragment> fragments) {

```

```

    super(fm);
    mFragments = fragments;
}
@Override
public Fragment getItem(int position) {
    return mFragments.get(position);
}
@Override
public int getCount() {
    return mFragments.size();
}
}

```

步骤 4：在引导页 Activity 中使用

```

public class GuideActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_guide);
        ViewPager viewPager = findViewById(R.id.view_pager);
        // 准备3个引导页Fragment
        List<Fragment> fragments = new ArrayList<>();
        fragments.add(GuideFragment.newInstance(R.drawable.guide1)); // 图片1
        fragments.add(GuideFragment.newInstance(R.drawable.guide2)); // 图片2
        fragments.add(new LastGuideFragment()); // 最后一页
        // 设置适配器
        viewPager.setAdapter(new GuidePagerAdapter(getSupportFragmentManager(), fragments
    ));
    }
}

```

六、案例：记账本 APP 实现

1. 功能需求

- 添加账单（金额、类型、日期、备注）；
- 展示账单列表（静态数据→动态数据）。

2. 准备工作：创建账单实体类

```
public class Bill {  
    private int id;  
    private double amount; // 金额  
    private String type; // 类型（如"餐饮"、"交通"）  
    private String date; // 日期  
    private String note; // 备注  
    // 构造方法  
    public Bill(int id, double amount, String type, String date, String note) {  
        this.id = id;  
        this.amount = amount;  
        this.type = type;  
        this.date = date;  
        this.note = note;  
    }  
    // getter和setter（略）  
}
```

3. 功能 1：添加账单（用 Fragment 实现）

步骤 1：添加账单的 Fragment 布局（fragment_add_bill.xml）

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    android:orientation="vertical"  
    android:padding="16dp">  
    <EditText  
        android:id="@+id/et_amount"  
        android:layout_width="match_parent"  
        android:layout_height="wrap_content"  
        android:hint="请输入金额"  
        android:inputType="numberDecimal" />  
    <EditText  
        android:id="@+id/et_type"
```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:hint="请输入类型（如餐饮、交通）" />
<EditText
    android:id="@+id/et_date"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="10dp"
    android:hint="请输入日期（如2023-10-01）" />
<EditText
    android:id="@+id/et_note"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="10dp"
    android:hint="请输入备注" />
<Button
    android:id="@+id/btn_save"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"
    android:text="保存账单" />
</LinearLayout>

```

步骤 2：添加账单的 Fragment 代码

```

public class AddBillFragment extends Fragment {
    private EditText etAmount, etType, etDate, etNote;
    @Nullable
    @Override
    public View onCreateView(...) {
        View view = inflater.inflate(R.layout.fragment_add_bill, container, false);
        etAmount = view.findViewById(R.id.et_amount);
        etType = view.findViewById(R.id.et_type);
        etDate = view.findViewById(R.id.et_date);
        etNote = view.findViewById(R.id.et_note);
        // 保存按钮点击事件
        view.findViewById(R.id.btn_save).setOnClickListener(v -> saveBill());
        return view;
    }
}

```

```

}
// 保存账单
private void saveBill() {
    // 获取输入内容
    String amountStr = etAmount.getText().toString();
    String type = etType.getText().toString();
    String date = etDate.getText().toString();
    String note = etNote.getText().toString();
    if (amountStr.isEmpty() || type.isEmpty() || date.isEmpty()) {
        Toast.makeText(getContext(), "请填写必填项", Toast.LENGTH_SHORT).show();
        return;
    }
    // 转换金额为double
    double amount = Double.parseDouble(amountStr);
    // 创建账单对象（实际项目中应存入数据库，这里先存到内存列表）
    Bill bill = new Bill(BillListFragment.billList.size() + 1, amount, type, date, note);
    BillListFragment.billList.add(bill); // 添加到全局列表
    // 提示保存成功
    Toast.makeText(getContext(), "保存成功", Toast.LENGTH_SHORT).show();
    // 清空输入
    etAmount.setText("");
    etType.setText("");
    etDate.setText("");
    etNote.setText("");
}
}

```

4. 功能 2：账单列表（静态→动态数据）

步骤 1：账单列表项布局（item_bill.xml）

```

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="80dp"
    android:gravity="center_vertical"
    android:padding="10dp">
    <TextView
        android:id="@+id/tv_type"

```

```

        android:layout_width="80dp"
        android:layout_height="wrap_content"
        android:text="餐饮"
        android:textSize="16sp" />
<TextView
    android:id="@+id/tv_amount"
    android:layout_width="100dp"
    android:layout_height="wrap_content"
    android:text="¥100"
    android:textColor="#FF0000" />
<TextView
    android:id="@+id/tv_date"
    android:layout_width="100dp"
    android:layout_height="wrap_content"
    android:text="2023-10-01" />
<TextView
    android:id="@+id/tv_note"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginStart="10dp"
    android:text="备注" />
</LinearLayout>

```

步骤 2：账单列表 Fragment（用 RecyclerView）

```

public class BillListFragment extends Fragment {
    // 全局账单列表（实际项目用数据库，这里简化）
    public static List<Bill> billList = new ArrayList<>();
    private RecyclerView recyclerView;
    private BillAdapter adapter;
    @Nullable
    @Override
    public View onCreateView(...) {
        View view = inflater.inflate(R.layout.fragment_bill_list, container, false);
        recyclerView = view.findViewById(R.id.recycler_bill);
        recyclerView.setLayoutManager(new LinearLayoutManager(getContext()));
        // 初始化静态数据（首次加载时）
        if (billList.isEmpty()) {
            initStaticData();
        }
    }
}

```

```

    }
    // 设置适配器
    adapter = new BillAdapter(billList);
    recyclerView.setAdapter(adapter);
    return view;
}
// 初始化静态数据
private void initStaticData() {
    billList.add(new Bill(1, 50.0, "餐饮", "2023-10-01", "早餐"));
    billList.add(new Bill(2, 200.0, "交通", "2023-10-02", "打车"));
}
// 账单适配器
class BillAdapter extends RecyclerView.Adapter<BillAdapter.ViewHolder> {
    private List<Bill> mBills;
    public BillAdapter(List<Bill> bills) {
        mBills = bills;
    }
    @Override
    public ViewHolder onCreateViewHolder(...) {
        View view = LayoutInflater.from(getContext())
            .inflate(R.layout.item_bill, parent, false);
        return new ViewHolder(view);
    }
    @Override
    public void onBindViewHolder(...) {
        Bill bill = mBills.get(position);
        holder.tvType.setText(bill.getType());
        holder.tvAmount.setText("¥" + bill.getAmount());
        holder.tvDate.setText(bill.getDate());
        holder.tvNote.setText(bill.getNote());
    }
    @Override
    public int getItemCount() {
        return mBills.size();
    }
}
class ViewHolder extends RecyclerView.ViewHolder {
    TextView tvType, tvAmount, tvDate, tvNote;
    public ViewHolder(View itemView) {
        super(itemView);
        tvType = itemView.findViewById(R.id.tv_type);
        tvAmount = itemView.findViewById(R.id.tv_amount);
    }
}

```

```

        tvDate = itemView.findViewById(R.id.tv_date);
        tvNote = itemView.findViewById(R.id.tv_note);
    }
}
}
}

```

步骤 3：主 Activity（切换添加 / 列表 Fragment）

```

public class BillActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_bill);
        // 初始显示账单列表Fragment
        getSupportFragmentManager().beginTransaction()
            .replace(R.id.container, new BillListFragment())
            .commit();
        // 底部按钮切换Fragment
        findViewById(R.id.btn_list).setOnClickListener(v -> {
            getSupportFragmentManager().beginTransaction()
                .replace(R.id.container, new BillListFragment())
                .commit();
        });
        findViewById(R.id.btn_add).setOnClickListener(v -> {
            getSupportFragmentManager().beginTransaction()
                .replace(R.id.container, new AddBillFragment())
                .commit();
        });
    }
}

```

总结

这节课咱们掌握了 Fragment 的核心用法和记账本案例：

1. Fragment 注册方式：

- 静态注册：在布局中用<fragment>标签，简单但不灵活；

- 动态注册：用FragmentManager和FragmentTransaction在代码中添加 / 替换，灵活常用。

2. **Fragment 生命周期**：重点关注onCreateView()（加载布局）和与 Activity 交互的onAttach()/onDetach()。

3. **记账本案例**：

- 用 Fragment 实现添加账单和账单列表两个页面；
- 静态数据：初始化时添加测试账单；
- 动态数据：添加账单后实时更新列表（通过全局列表暂存，实际项目用数据库）。

Fragment 是实现复杂界面的基础，尤其是在平板 APP 中应用广泛。课后可以给记账本添加 "删除账单" 和 "用 Room 持久化存储" 功能，巩固所学。有问题随时问～

（注：文档部分内容可能由 AI 生成）